Memory Networks for Question Answering: Mathematical Foundations, Relation Extraction, and Graph-Based Reasoning

[Student Name]

[Department], [University]

[Course Number]: [Course Name]

[Instructor Name]

October 9, 2026

Abstract

Memory networks (Weston et al., 2015) pair a learning model with a long-term memory that the model can write to and read from. This paper explains how they work. Part A covers the memory, the four components *I*, *G*, *O*, and *R*, the flow of data through the model, the text implementation called MemNN, and its training with a margin ranking loss. Part B shows how a pre-trained relation extractor, REBEL (Huguet Cabot & Navigli, 2021), can turn a question and its answer choices into subject–relation–object triplets. Part C uses the question “Why did the engine stop?” and the choice “The fuel tank was empty.” to show how those triplets become a small graph, how a relational graph neural network walks the path from the question to the answer, and how the question text is fused with the graph. The paper ends by linking the two ideas: a graph neural network over retrieved facts does the same job as the *O* module of a memory network, but it follows labeled relations instead of guessing which stored sentences belong together.

Keywords: memory networks, question answering, relation extraction, knowledge graphs, graph neural networks

Memory Networks for Question Answering: Mathematical Foundations, Relation Extraction, and Graph-Based Reasoning

To answer a question about a story, a system must remember the facts in the story and then combine the right ones. Recurrent neural networks (RNNs) and long short-term memory networks (LSTMs; Hochreiter & Schmidhuber, 1997) store everything they have read in one hidden state vector. That vector is small, and old facts fade or get overwritten. Weston et al. (2015) proposed *memory networks* to fix this. A memory network keeps an explicit memory, a list of stored items, and learns how to write to it, how to look things up in it, and how to turn what it found into an answer.

This paper has three parts. Part A explains the model in the original paper and the math behind it. Part B shows how a pre-trained relation extractor can turn questions and answer choices into structured triplets. Part C shows how those triplets can form a small graph and how a graph neural network (GNN) can reason over it, using the question “Why did the engine stop?” as a running example. Runnable Python versions of the Part B and Part C examples come with this paper (files `relation_extraction_demo.py` and `graph_reasoning_demo.py`).

Part A: Memory Networks

Weston et al. (2015) describe memory networks first as a general framework and then as one concrete neural version for text, called a MemNN. The framework says *what* the parts must do. The MemNN says *how* one simple version does it.

Memory

The memory **m** is an array of objects indexed by *i*, written $\mathbf{m}_i$. The objects can be vectors or raw strings (Weston et al., 2015, p. 1). In the text version, each slot holds one sentence in its original form, such as “Joe went to the kitchen.” The memory has three key properties.

- **It is compartmentalized.** Each fact sits in its own slot. This differs from an RNN, which compresses all facts into one dense vector, and from classic associative memories, which spread knowledge across all the weights of a network (Weston et al., 2015, p. 6).

- **It can be read and written.** The model adds new facts at both training and test time. The model *parameters* stay fixed at test time, but the *memory contents* keep changing. This is how the model can answer questions about a story it has never seen.
- **It can be very large.** In the paper's large-scale experiment, the memory holds 14 million facts, stored as (subject, relation, object) triples from the REVERB extractions used by Fader et al. (2013).

The paper also discusses how to manage a large memory. A slot-choosing function H decides where each new item goes. H could group memories by topic or entity, so later lookups only search the right group. If the memory fills up, H could also "forget" by overwriting the least useful slot, though the authors did not test this.

Components

A memory network has four components. Each can be learned, and each can, in principle, use any machine learning method (Weston et al., 2015, p. 2).

- ***I* (input feature map)** turns the raw input x into an internal representation $I(x)$. For text, this can include parsing, coreference resolution, or mapping words to a bag-of-words vector.
- ***G* (generalization)** updates the memory using the new input. The simplest G stores $I(x)$ in a free slot and leaves all other slots alone:

$$\mathbf{m}_{H(x)} = I(x) \tag{1}$$

where $H(x)$ picks the slot. A richer G could revise old memories based on new evidence. The authors call it "generalization" because this is where the network could compress and generalize what it stores.

- ***O* (output feature map)** reads the memory. Given the input and the current memory, it finds the relevant memories and produces output features o. This is where inference happens.

- ***R* (response)** turns *o* into the final answer, such as a single word, a sentence produced by an RNN, or an action.

Flow of the Model

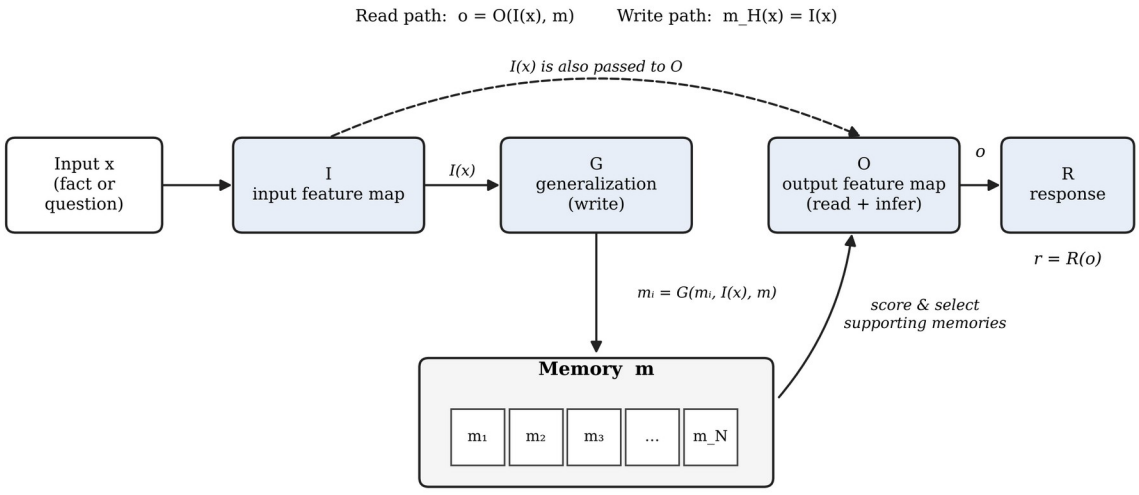
Given an input *x* (a word, a sentence, an image, and so on), the model runs four steps (Weston et al., 2015, p. 2):

1. Convert *x* to an internal feature representation $I(x)$.
2. Update the memories given the new input: $\mathbf{m}_i = G(\mathbf{m}_i, I(x), \mathbf{m})$ for all i.
3. Compute output features from the input and the memory: $o = O(I(x), \mathbf{m})$.
4. Decode the output features into a response: $r = R(o)$.

Figure 1 shows this flow. A statement such as “Joe went to the kitchen” mostly uses the write path (*I* then *G*). A question such as “Where is the milk now?” mostly uses the read path (*I*, then *O*, then *R*). The same steps run during training and at test time.

Figure 1

Data Flow Through a Memory Network



Note. *I*, *G*, *O*, and *R* are the four components defined by Weston et al. (2015). *G* writes the encoded input into memory; *O* scores memories against the input and selects supporting facts; *R* produces the answer. Figure created by the author.

Implementation of a Memory Network for Text

A memory network whose components are neural networks is called a memory neural network, or MemNN. Section 3 of the paper builds a simple MemNN for text (Weston et al., 2015, pp. 2–6).

Input and Generalization

I takes a sentence, which is either a fact or a question. *G* stores it in the next empty slot, so $\mathbf{m}_N = x$ and then $N = N + 1$. Old memories are never changed.

Output: Finding Supporting Memories

The core of the model is the *O* module. It finds *k* supporting memories for the input *x*; the paper uses *k* up to 2. The first supporting memory is the one that best matches the question:

$$o_1 = O_1(x, \mathbf{m}) = \arg\max_{i=1,\ldots,N} s_O(x, \mathbf{m}_i) \tag{2}$$

where s_O is a learned scoring function. For $k = 2$, the model looks for a second memory that matches both the question and the first memory found:

$$o_2 = O_2(x, \mathbf{m}) = \arg\max_{i=1,\ldots,N} s_O([x, \mathbf{m}_{o_1}], \mathbf{m}_i) \tag{3}$$

The square brackets denote a list. Because inputs are bags of words, scoring the list is the same as adding $s_O(x, \mathbf{m}_i)$ and $s_O(\mathbf{m}_{o1}, \mathbf{m}_i)$. The output is $o = [x, \mathbf{m}_{o1}, \mathbf{m}_{o2}]$. This two-step search is called a *hop*. Each hop lets the model chain one more fact.

Response

The simplest *R* returns the retrieved sentence $\mathbf{m}_{ok}$. The paper's main version instead returns a single word by ranking every word *w* in the dictionary *W*:

$$r = \arg\max_{w \in W} s_R([x, \mathbf{m}_{o_1}, \mathbf{m}_{o_2}], w) \tag{4}$$

An RNN or LSTM can replace this step when full sentences are needed.

Scoring Function

Both s_O and s_R have the same form, an embedding model:

$$s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y) \tag{5}$$

Here Φ_x and Φ_y map text to a D-dimensional sparse feature vector (a bag of words), and U is an $n \times D$ matrix that projects those features into an n-dimensional embedding space. The score is the dot product of the two embeddings, $U\Phi_x(x)$ and $U\Phi_y(y)$. The paper sets $D = 3|W|$. Each word gets three separate representations: one when it appears in the candidate y, and two for the input side, depending on whether it comes from the question or from an already retrieved memory. s_O and s_R use different matrices, U_O and U_R.

A Worked Example

Take the story in Figure 1 of the original paper: “Joe went to the kitchen. Fred went to the kitchen. Joe picked up the milk. Joe travelled to the office. Joe left the milk. Joe went to the bathroom.” The question is x = “Where is the milk now?”

- Hop 1 (Equation 2): the best match for x is $\mathbf{m}_{o1}$ = “Joe left the milk,” because it shares the word *milk*.
- Hop 2 (Equation 3): given [x, $\mathbf{m}_{o1}$], the best match is $\mathbf{m}_{o2}$ = “Joe travelled to the office,” the last place Joe went before leaving the milk.
- Response (Equation 4): ranking all words given [x, $\mathbf{m}_{o1}$, $\mathbf{m}_{o2}$] gives r = “office.”

Note that “office” does not appear in the sentence that mentions the milk. The model reaches it only by chaining two facts. This is what “reasoning over memory” means here.

Extensions

The paper adds four extensions to the basic model (Weston et al., 2015, pp. 4–6).

- **Word sequences as input.** When text arrives as a stream of words, a learned segmenter decides where a statement ends: $seg(c) = W_{seg}{}^{T} U_S \Phi_{seg}(c)$. If the score passes the margin γ, the words so far are written to memory as one slot.
- **Hashing for speed.** Scoring 14 million memories for each question is slow. The paper hashes the input into buckets and only scores memories in the same buckets. Hashing by word is fast but misses memories with no shared word. Hashing by clusters of word embeddings (K-means on the columns of U_O) also catches synonyms.

- **Write time.** In stories, the order of facts matters. Instead of scoring one memory at a time, the model compares two candidate memories y and y':

$$s_{O_t}(x, y, y') = \Phi_x(x)^\top U_{O_t}^\top U_{O_t}\left(\Phi_y(y) - \Phi_y(y') + \Phi_t(x, y, y')\right) \tag{6}$$

Φ_t adds three binary features that say whether x is older than y, whether x is older than y', and whether y is older than y'. A positive score means y is preferred. The argmax becomes a loop that keeps the current winner and compares it with each next memory.

- **Unseen words and exact matches.** A new word (the paper uses "Boromir") is represented by the words that appear around it, which raises D to $5|W|$. Because low-dimensional embeddings blur exact word matches, the paper also adds matching features that mark words shared by x and y (giving $D = 8|W|$), or mixes in a raw bag-of-words score:

$$\Phi_x(x)^\top U^\top U \Phi_y(y) + \lambda\, \Phi_x(x)^\top \Phi_y(y) \tag{7}$$

Training Procedure for a Memory Network

The MemNN is trained with full supervision. For each training question, the data give the correct answer r and also mark which sentences are the supporting facts, $\mathbf{m}_{o1}$ and $\mathbf{m}_{o2}$. The supporting facts are *not* given at test time. Knowing the right choices for both argmax steps during training allows a simple loss. The model minimizes, over the parameters U_O and U_R, a sum of three margin ranking (hinge) terms (Weston et al., 2015, p. 4):

$$\sum_{\bar{f} \neq \mathbf{m}_{o_1}} \max\left(0,\ \gamma - s_O(x, \mathbf{m}_{o_1}) + s_O(x, \bar{f})\right) \tag{8a}$$

$$+ \sum_{\bar{f}' \neq \mathbf{m}_{o_2}} \max\left(0,\ \gamma - s_O([x, \mathbf{m}_{o_1}], \mathbf{m}_{o_2}) + s_O([x, \mathbf{m}_{o_1}], \bar{f}')\right) \tag{8b}$$

$$+ \sum_{\bar{r} \neq r} \max\left(0,\ \gamma - s_R([x, \mathbf{m}_{o_1}, \mathbf{m}_{o_2}], r) + s_R([x, \mathbf{m}_{o_1}, \mathbf{m}_{o_2}], \bar{r})\right) \tag{8c}$$

Here $\bar{f}$, $\bar{f}'$, and $\bar{r}$ are wrong choices and γ is the margin. Each term says the same thing: the correct choice must score at least γ higher than any wrong choice. If it already does, the term is zero and nothing changes. If it does not, the gradient pushes the correct score up and the wrong

score down. Term 8a trains the first hop, term 8b trains the second hop, and term 8c trains the answer ranking.

The model is trained with stochastic gradient descent (SGD). Summing over every wrong choice would be expensive, so at each step the model samples one $\bar{f}$, $\bar{f}'$, and $\bar{r}$, as in the WSABIE method of Weston et al. (2011). In the simulated-world experiments, the authors used an embedding size of 100, a learning rate of 0.01, a margin γ of 0.1, and 10 training epochs. When an RNN serves as R, term 8c is replaced by the usual log-likelihood loss for language modeling on the sequence $[x, o_1, o_2, r]$. The segmenter and the write-time model have their own hinge losses of the same kind (Weston et al., 2015, Appendices B and C).

Table 1 summarizes the main results. On the large-scale QA task, the MemNN matched earlier embedding methods, and adding bag-of-words features raised F1 from 0.72 to 0.82. Cluster hashing gave about an 80-fold speedup with almost no loss. On the simulated stories, the version with two hops and time features answered nearly every question, while RNNs and LSTMs failed on the harder tasks.

Table 1

Selected Results Reported by Weston et al. (2015)

Task	Model	Result
Large-scale QA (Fader et al., 2013)	Fader et al. (2013)	F1 = 0.54
	Bordes et al. (2014)	F1 = 0.73
	MemNN, embedding only	F1 = 0.72
	MemNN, with bag-of-words features	F1 = 0.82
	MemNN, cluster hash (80× faster)	F1 = 0.80
Simulated world, difficulty 5, actor + object	RNN	17.8% accuracy
	LSTM	29.0% accuracy
	MemNN, $k = 1$	18.5% accuracy
	MemNN, $k = 2$, with time features	99.9% accuracy

Note. Values from Tables 1, 2, and 3 of Weston et al. (2015). “Difficulty 5” means the needed fact may be up to five steps back in the story.

Later work kept the idea but changed the training. End-to-end memory networks replace the hard argmax with a soft attention over all memories, so the model can learn from answers alone, without labels for supporting facts (Sukhbaatar et al., 2015). Dynamic memory networks add an episodic memory module with gated recurrent updates (Kumar et al., 2016). The bAbI tasks, built from the same kind of simulated world, became a standard test for this family of models (Weston et al., 2016).

Part B: Relation Extraction

Relation extraction turns free text into triplets of the form (subject, relation, object). Early systems used hand-written patterns or open extraction, which keeps the verb phrase as the relation (Banko et al., 2007). Later systems used distant supervision, which labels training sentences by matching them to facts already in a knowledge base (Mintz et al., 2009). Modern pre-trained models learn the whole task end to end. REBEL (Huguet Cabot & Navigli, 2021) is one such model. It is a sequence-to-sequence transformer (BART) fine-tuned to read a sentence and write out its triplets in a fixed format that uses over 200 relation types from Wikidata:

```
<triplet> subject <subj> object <obj> relation
```

This link to memory networks is direct. The large-scale experiment in Weston et al. (2015) stored 14 million triplets that an extraction system had already pulled from the web. Relation extraction is how those memories are made.

Step 1: Prepare the Inputs

A question and its answer choices are short and often do not state a full fact. “Why did the engine stop?” asks for a cause; it does not state one. For this reason, the example runs the extractor on three kinds of text: (a) the question alone, (b) each answer choice alone, and (c) one declarative *hypothesis* per choice that joins the two, such as “The engine stopped because the

fuel tank was empty.” The question word also gives a cue: *why* signals that the missing link is a cause.

Step 2: Run the Pre-trained Extractor

The code below loads REBEL through the Hugging Face `transformers` library and parses its output into Python tuples. REBEL’s special tokens must be kept when decoding, because they mark where each part of a triplet starts.

```
def parse_rebel(text):
    """REBEL emits '<triplet> subject <subj> object <obj> relation ...'."""
    triplets, subj, obj, rel, state = [], "", "", "", None
    text = text.replace("<s>", "").replace("<pad>", "").replace("</s>", "")
    for tok in text.split():
        if tok == "<triplet>":
            if rel:
                triplets.append((subj.strip(), rel.strip(), obj.strip()))
            subj, obj, rel, state = "", "", "", "subj"
        elif tok == "<subj>":
            if rel:                      # same subject, new object/relation
                triplets.append((subj.strip(), rel.strip(), obj.strip()))
            obj, rel, state = "", "", "obj"
        elif tok == "<obj>":
            state = "rel"
        elif state == "subj":
            subj += " " + tok
        elif state == "obj":
            obj += " " + tok
        elif state == "rel":
            rel += " " + tok
    if subj and rel and obj:
        triplets.append((subj.strip(), rel.strip(), obj.strip()))
    return triplets

if __name__ == "__main__":
    from transformers import pipeline

    extractor = pipeline("text2text-generation", model="Babelscape/rebel-large",
                         tokenizer="Babelscape/rebel-large")
    question = "Why did the engine stop?"
    choices = ["The fuel tank was empty.", "The driver turned on the radio.",
               "The tires were new."]
    texts = [question] + choices + [to_statement(question, c) for c in choices]
    for t in texts:
        out = extractor(t, return_tensors=True, return_text=False)
        ids = out[0]["generated_token_ids"]
        decoded = extractor.tokenizer.batch_decode([ids])[0]
        print(f"{t!r:70} -> {parse_rebel(decoded)}")
```

Step 3: Read and Normalize the Triplets

Table 2 shows the kind of output this pipeline gives for the example. Raw relation names from REBEL (Wikidata labels such as *has cause* and *has effect*) and verb phrases from an open extractor are then mapped to a small, fixed set of relations that the graph in Part C uses. Mapping to the relation names of ConceptNet (Speer et al., 2017) makes it easy to link the triplets to background knowledge in the next step.

Table 2

Triplets Extracted From the Question and Answer Choices

Source text	Raw triplet (subject | relation | object)	Normalized triplet
Q: Why did the engine stop?	engine | has effect | stop	(engine, HasSubevent, stop); target relation = Causes, object = ?
A: The fuel tank was empty.	fuel tank | has quality | empty	(fuel tank, HasState, empty)
B: The driver turned on the radio.	driver | uses | radio	(driver, Uses, radio)
C: The tires were new.	tires | has quality | new	(tires, HasProperty, new)
Hypothesis A: The engine stopped because the fuel tank was empty.	fuel tank was empty | has effect | engine stopped	claim: (empty fuel tank, Causes, engine stop)
Hypothesis B: The engine stopped because the driver turned on the radio.	radio | has effect | engine stopped	claim: (radio, Causes, engine stop)

Note. Exact strings depend on the model version and decoding settings; the rows show the expected shape of the output. Hypothesis rows are claims to be checked, not evidence.

One point in Table 2 needs care. The extractor copies whatever a sentence claims. It pulls a causal triplet from Hypothesis B just as readily as from Hypothesis A, even though turning on a radio does not stop an engine. So the hypothesis triplets cannot decide the answer by themselves. Their job is to name the entities and the relation the answer must support. The evidence must come from somewhere else, namely a knowledge graph of background facts, and the next part uses a GNN to test whether that evidence connects the question to each choice.

Step 4: Ground the Entities in a Knowledge Graph

Each extracted subject and object is matched to a node in a commonsense knowledge graph, such as ConceptNet (Speer et al., 2017) or ATOMIC, which stores everyday cause-and-effect knowledge (Sap et al., 2019). This is the grounding step used by KagNet (Lin et al., 2019) and QA-GNN (Yasunaga et al., 2021). For the example, grounding finds *engine* and *stop* for the question, *fuel tank* and *empty* for choice A, and pulls in neighboring background facts such as (engine, Uses, fuel), (fuel tank, Contains, fuel), (empty, Implies, lack of fuel), and (lack of fuel, Causes, stop). These are the edges of the sub-graph in Part C.

Part C: Graph-Based Reasoning

The question is “Why did the engine stop?” and the answer choice is “The fuel tank was empty.” The two sentences share no content words. A system that only matches words, like the basic MemNN scoring function in Equation 5, has nothing to match. The link between them exists only in background knowledge: an engine needs fuel, a fuel tank holds fuel, an empty tank means there is no fuel, and no fuel makes an engine stop. A graph makes this chain explicit, and a GNN can learn to follow it.

Building the Sub-Graph

The sub-graph is built in four steps, following Lin et al. (2019) and Yasunaga et al. (2021).

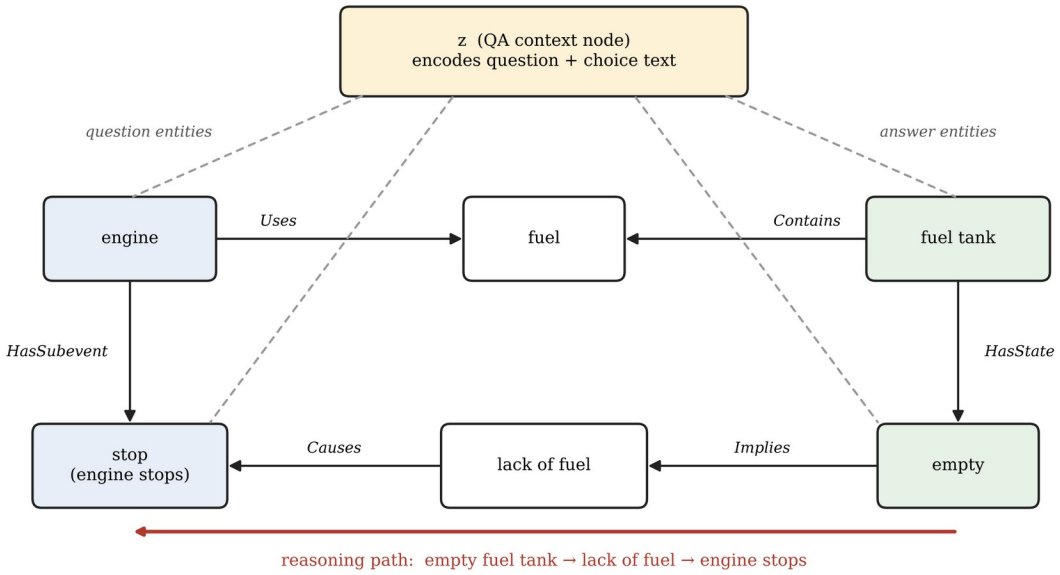
1. **Entity nodes.** Add one node for each grounded entity from Part B. Question entities are *engine* and *stop*; answer entities are *fuel tank* and *empty*.
2. **Bridge nodes.** Add knowledge-graph nodes that lie on short paths (one or two hops) between question and answer entities. Here these are *fuel* and *lack of fuel*.
3. **Typed edges.** Add every triplet that links two nodes in the set, keeping its relation type: HasSubevent, Uses, Contains, HasState, Implies, and Causes. Each edge is also added in the reverse direction with its own relation type, so messages can flow both ways.

4. **Context node.** Add one extra node, *z*, that stands for the text of the question and the choice. Connect *z* to every question and answer entity with a Context edge. This node is how the text enters the graph (see the final section).

Figure 2 shows the result. A separate sub-graph is built for each answer choice. For choice B ("The driver turned on the radio"), the only links between the radio and the engine run through weak PartOf edges (both are part of a car), and no Causes edge reaches *stop*.

Figure 2

Sub-Graph for "Why Did the Engine Stop?" and "The Fuel Tank Was Empty."



Note. Blue nodes come from the question, green nodes from the answer choice, and white nodes are bridge concepts from the knowledge graph. Dashed lines connect the text context node *z* to each grounded entity. The red arrow marks the path the GNN should learn to follow. Figure created by the author.

Walking the Path With a Graph Neural Network

A GNN computes a vector for each node by passing messages along edges (Scarselli et al., 2009; Gilmer et al., 2017). After *L* rounds, each node's vector holds information from nodes

up to L hops away. Because our edges have types, we use a relational graph convolutional network (R-GCN; Schlichtkrull et al., 2018), which learns one weight matrix per relation type, and we add attention over incoming edges, as in graph attention networks (Veličković et al., 2018). This combination is close to the GNN used in QA-GNN (Yasunaga et al., 2021).

Node Initialization

Each entity node v starts with an embedding of its name from a text encoder, $h_v^{(0)} = e(\text{name}_v)$. The context node starts with the text vector z, defined in Equation 14 below.

Message Passing

At layer l, each node v collects messages from its neighbors u. A message is the neighbor's vector multiplied by the weight matrix of the edge's relation type r, and each message is weighted by an attention score α:

$$h_v^{(l+1)} = \text{ReLU}\left(W_0^{(l)} h_v^{(l)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_v^r} \alpha_{uv}^{(l)} \, W_r^{(l)} h_u^{(l)} \right) \tag{9}$$

$$\alpha_{uv}^{(l)} = \text{softmax}_u\left(\frac{(W_q h_v^{(l)})^\top \, W_k \, [h_u^{(l)}; e_r]}{\sqrt{d}} \right) \tag{10}$$

Here $\mathcal{N}_v^r$ is the set of neighbors linked to v by relation r, e_r is a learned embedding of the relation, d is the vector size, and the softmax runs over all edges coming into v. The relation matrices W_r let the model treat a Causes edge very differently from a PartOf edge. The attention α lets each node decide which neighbor matters most for this question.

Walking the Path

Each layer moves information one hop. The path from the answer to the question in Figure 2 has two hops (*empty → lack of fuel → stop*), plus one hop from z, so $L = 3$ layers are enough. Table 3 traces what happens, layer by layer, in a trained model.

Table 3

How Information Moves Along the Path, Layer by Layer

Layer	Edge followed	What the receiving node learns
1	empty → lack of fuel (Implies)	*lack of fuel* now carries the fact that the tank is empty.

Layer	Edge followed	What the receiving node learns
1	fuel tank → empty (HasState)	*empty* is tied to the fuel tank, not to some other container.
2	lack of fuel → stop (Causes)	*stop* now carries "empty tank → no fuel → causes stop."
2	engine → stop (HasSubevent)	*stop* is tied to the engine, the subject of the question.
3	stop → z (Context, reverse)	*z* receives the full causal chain and can compare it with the question.

Note. In a trained model, attention α is high on the Causes and Implies edges for a "why" question and low on edges such as PartOf.

After training, the path can also be read back out, which makes the answer explainable. For a path $\pi = (v_0, v_1, \ldots, v_k)$ from an answer entity to a question entity, the path score is the sum of the log attention weights along it, averaged over layers:

$$\text{score}(\pi) = \sum_{j=1}^{k} \log \bar{\alpha}_{v_{j-1} v_j} \tag{11}$$

The highest-scoring path is the explanation. For choice A it is *empty → lack of fuel → stop*, which matches the human reasoning. KagNet (Lin et al., 2019) and MHGRN (Feng et al., 2020) score paths in a similar way.

Scoring and Training

For each choice *a*, the model pools the node vectors with attention guided by the text, then scores the choice with a small feed-forward network:

$$g = \sum_{v} \beta_v \, h_v^{(L)}, \qquad \beta_v = \text{softmax}_v\left(z^{\top} h_v^{(L)}\right) \tag{12}$$

$$\text{score}(q, a) = \text{MLP}\left([z;\ h_z^{(L)};\ g]\right), \qquad P(a \mid q) = \frac{\exp \text{ score}(q, a)}{\sum_{a'} \exp \text{ score}(q, a')} \tag{13}$$

The model is trained with cross-entropy, $-\log P(a^* \mid q)$, where a^* is the correct choice. Unlike the original MemNN, this needs no labels for the supporting facts. The model learns which edges to attend to from the answer labels alone, the same move that end-to-end memory networks made (Sukhbaatar et al., 2015).

Example Code

The code below is a compact PyTorch version of Equations 9 to 13. The full script, `graph_reasoning_demo.py`, also builds the graph for each choice, runs one training step, and extracts the best path. It was run to check that it works end to end. Because the demo uses a stand-in text encoder and is not trained, its probabilities are close to one third each; it only shows the mechanics.

```
class RelAttnLayer(nn.Module):
    """h_v' = ReLU(W0 h_v + sum_u alpha_uv W_r h_u), alpha = softmax over in-edges."""

    def __init__(self, d, n_rel):
        super().__init__()
        self.W0 = nn.Linear(d, d)
        self.W_rel = nn.Parameter(torch.randn(n_rel, d, d) * d ** -0.5)
        self.rel_emb = nn.Embedding(n_rel, d)
        self.q = nn.Linear(d, d)
        self.k = nn.Linear(2 * d, d)

    def forward(self, h, src, dst, rel):
        msg = torch.einsum("ed,edk->ek", h[src], self.W_rel[rel])      # W_r h_u
        key = self.k(torch.cat([h[src], self.rel_emb(rel)], dim=-1))
        score = (self.q(h)[dst] * key).sum(-1) / h.size(1) ** 0.5
        alpha = torch.zeros_like(score)
        for v in dst.unique():                # softmax over each node's in-edges
            m = dst == v
            alpha[m] = F.softmax(score[m], dim=0)
        agg = torch.zeros_like(h).index_add_(0, dst, alpha.unsqueeze(-1) * msg)
        return F.relu(self.W0(h) + agg), alpha

class GraphQA(nn.Module):
    def __init__(self, d=D, n_rel=N_REL, layers=3):
        super().__init__()
        self.layers = nn.ModuleList(RelAttnLayer(d, n_rel) for _ in range(layers))
        self.out = nn.Sequential(nn.Linear(3 * d, d), nn.ReLU(), nn.Linear(d, 1))

    def forward(self, z, nodes, src, dst, rel):
        h = torch.stack([z] + [encode(n) for n in nodes[1:]])  # node 0 = text
        alphas = []
        for layer in self.layers:
            h, a = layer(h, src, dst, rel)
            alphas.append(a)
        att = F.softmax(h[1:] @ z, dim=0)     # text-guided pooling of graph nodes
        g = (att.unsqueeze(-1) * h[1:]).sum(0)
        return self.out(torch.cat([z, h[0], g])).squeeze(), alphas
```

Combining the Question Text With the Extracted Relations

The graph alone is not enough. It knows that empty tanks cause engines to stop, but it does not know which question was asked. The text alone is not enough either. A language model may not hold the fuel fact reliably, and it cannot show its reasoning. The model therefore fuses the two at four points.

1. **Encode the question and choice together.** The question and the choice are joined into one input and passed through a pre-trained encoder such as BERT (Devlin et al., 2019). The vector at the [CLS] position is the text representation:

$$z = f_{\mathrm{LM}}([\mathrm{CLS}]\ q\ [\mathrm{SEP}]\ a\ [\mathrm{SEP}]) \tag{14}$$

Joining q and a lets the encoder's self-attention compare every word of the question with every word of the choice before the graph is used.

2. **Put the text inside the graph.** The vector z becomes the starting state of the context node, which is linked to every question and answer entity (Figure 2). During message passing, the text sends information into the graph and the graph sends the reasoning chain back into z. This is the "joint graph" design of QA-GNN (Yasunaga et al., 2021).
3. **Use the text to prune and weight the graph.** Knowledge graphs return many loosely related nodes. Each node v gets a relevance score from the encoder, $\rho_v = f_{LM}([q; a; \mathrm{name}_v])$, and low-scoring nodes are dropped or down-weighted. The attention in Equation 10 is also driven by the text, since the query vector comes from nodes that already carry z.
4. **Combine at the output.** The final score in Equation 13 uses z, the updated context node $h_z^{(L)}$, and the pooled graph vector g. The text and the graph each add evidence, and the network learns how much to trust each.

The triplets enter through two routes. Their entities set the graph's nodes, and their relations set the edge types, each with its own matrix W_r and embedding e_r. A simpler option is also possible: write each triplet out as a short sentence ("empty implies lack of fuel; lack of fuel

causes stop") and append it to the text input of Equation 14. This is easy to set up, but it loses the explicit graph structure and the readable path.

Connecting Back to Memory Networks

The two halves of this paper describe the same idea at two levels. In a memory network, the memory holds facts and the *O* module chains them hop by hop with Equations 2 and 3. In the graph approach, the extracted triplets *are* the memory, and each GNN layer is one hop. The difference is how the chaining happens. A MemNN must discover which memories belong together by learning word similarity. A GNN is told how facts connect through typed edges, so it can follow a Causes edge even when the two sentences share no words. This is why the graph approach handles "Why did the engine stop?" well, and why the plain word-matching s_O of Equation 5 would struggle with it. The memory network idea—keep facts in an explicit store and learn to read them in several hops—remains the foundation of both.

Conclusion

Memory networks give a learning model a store of facts it can write to and read from. The MemNN for text keeps each sentence in its own slot, scores memories with an embedding model, chains up to two facts per question, and learns with a margin ranking loss that needs labeled supporting facts. Relation extraction turns questions and answer choices into triplets that can serve as memories or as the edges of a graph. A relational GNN over a small sub-graph can then walk from "The fuel tank was empty" to "Why did the engine stop?" through *lack of fuel*, while a context node and a joint text encoder keep the reasoning tied to the actual question. The path the GNN follows also serves as an explanation of its answer.

References

Banko, M., Cafarella, M. J., Soderland, S., Broadhead, M., & Etzioni, O. (2007). Open information extraction from the web. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence* (pp. 2670–2676). IJCAI.

Bordes, A., Chopra, S., & Weston, J. (2014). Question answering with subgraph embeddings. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing* (pp. 615–620). Association for Computational Linguistics. https://doi.org/10.3115/v1/D14-1067

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Fader, A., Zettlemoyer, L., & Etzioni, O. (2013). Paraphrase-driven learning for open question answering. In *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics* (pp. 1608–1618). Association for Computational Linguistics.

Feng, Y., Chen, X., Lin, B. Y., Wang, P., Yan, J., & Ren, X. (2020). Scalable multi-hop relational reasoning for knowledge-aware question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 1295–1309). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.99

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1263–1272). PMLR.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Huguet Cabot, P.-L., & Navigli, R. (2021). REBEL: Relation extraction by end-to-end language generation. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 2370–2381). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.findings-emnlp.204

Kumar, A., Irsoy, O., Ondruska, P., Iyyer, M., Bradbury, J., Gulrajani, I., Zhong, V., Paulus, R., & Socher, R. (2016). Ask me anything: Dynamic memory networks for natural language processing. In *Proceedings of the 33rd International Conference on Machine Learning* (pp. 1378–1387). PMLR.

Lin, B. Y., Chen, X., Chen, J., & Ren, X. (2019). KagNet: Knowledge-aware graph networks for commonsense reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 2829–2839). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1282

Mintz, M., Bills, S., Snow, R., & Jurafsky, D. (2009). Distant supervision for relation extraction without labeled data. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP* (pp. 1003–1011). Association for Computational Linguistics.

Sap, M., Le Bras, R., Allaway, E., Bhagavatula, C., Lourie, N., Rashkin, H., Roof, B., Smith, N. A., & Choi, Y. (2019). ATOMIC: An atlas of machine commonsense for if-then reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, *33*(1), 3027–3035. https://doi.org/10.1609/aaai.v33i01.33013027

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). The graph neural network model. *IEEE Transactions on Neural Networks, 20*(1), 61–80. https://doi.org/10.1109/TNN.2008.2005605

Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). Modeling relational data with graph convolutional networks. In A. Gangemi et al. (Eds.),

The semantic web: ESWC 2018 (pp. 593–607). Springer. https://doi.org/10.1007/978-3-319-93417-4_38

Speer, R., Chin, J., & Havasi, C. (2017). ConceptNet 5.5: An open multilingual graph of general knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence, 31*(1), 4444–4451. https://doi.org/10.1609/aaai.v31i1.11164

Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2015). End-to-end memory networks. In *Advances in Neural Information Processing Systems 28* (pp. 2440–2448). Curran Associates.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations*. https://arxiv.org/abs/1710.10903

Weston, J., Bengio, S., & Usunier, N. (2011). WSABIE: Scaling up to large vocabulary image annotation. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence* (pp. 2764–2770). AAAI Press.

Weston, J., Bordes, A., Chopra, S., Rush, A. M., van Merriënboer, B., Joulin, A., & Mikolov, T. (2016). Towards AI-complete question answering: A set of prerequisite toy tasks. In *Proceedings of the 4th International Conference on Learning Representations*. https://arxiv.org/abs/1502.05698

Weston, J., Chopra, S., & Bordes, A. (2015). Memory networks. In *Proceedings of the 3rd International Conference on Learning Representations*. https://arxiv.org/abs/1410.3916

Yasunaga, M., Ren, H., Bosselut, A., Liang, P., & Leskovec, J. (2021). QA-GNN: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 535–546). Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.naacl-main.45